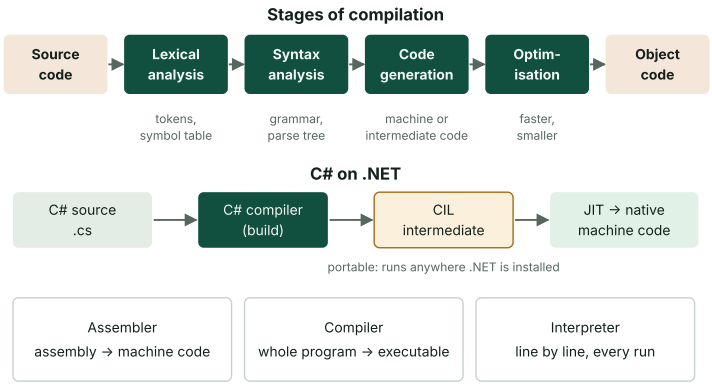
Translators, application software and processing systems

FD15 FACT FILE

Specification: students should be able to

- Describe the purpose of assemblers, compilers and interpreters, the stages of compilation and the use of intermediate code
- Explain what is meant by application software and the difference between generic and special purpose software
- Compare the use of custom-built software with off-the-shelf software
- Explain how application software can be delivered on different platforms, for example downloaded to a device or accessed using the web
- Evaluate different methods of obtaining software: proprietary and open source
- Describe batch processing and real-time processing and evaluate their use for common applications
- Describe the input, output and processing involved in ATMs, point of sale (POS) terminals and smartphones

Key words: compiler, interpreter, assembler, intermediate code, lexical analysis, application software, generic software, special purpose software, custom-built (bespoke) software, off-the-shelf software, platform, web application, proprietary software, open source software, batch processing, real-time processing, control total, EFTPOS



The stages of compilation from source code to object code, and how C# is compiled to Common Intermediate Language and then just-in-time compiled to machine code

Overview: from translators to processing systems

This lesson follows software from the moment it is written to the moment it is used. Programs written in high-level languages must be translated before a processor can run them, by a compiler, an interpreter, or a compiler that produces intermediate code for a virtual machine. The result is application software: programs that let users carry out their own tasks, such as writing a report or booking a ticket, and which reach the hardware only through the operating system. Application software can be generic, like a spreadsheet that serves countless purposes, or special purpose, built for one kind of job. An organisation can commission custom-built software or buy an off-the-shelf package, and it can install software on its own devices or use it through a web browser. It must also choose between proprietary software, owned and controlled by a company, and open source software, whose code anyone can examine and change. Finally, software processes data in different ways: batch processing collects transactions and handles them together later, while real-time processing deals with each one immediately. Everyday devices such as cash machines, shop tills and smartphones bring all of these ideas together as input, processing and output systems.

Translators and compilation

An assembler converts assembly language mnemonics into machine code, mostly one instruction for one instruction. A compiler translates a whole high-level program before it runs, producing an executable that runs quickly without the compiler and keeps the source code private; all the errors are listed together at the end. An interpreter translates and executes one statement at a time, stopping at the first error, which helps during development, but the program runs more slowly and needs the interpreter and the source code every time. Compilation passes through ordered stages. Lexical analysis removes comments and white space, splits the code into tokens and builds a symbol table of identifiers. Syntax analysis checks the tokens against the grammar of the language and builds a parse tree. Semantic checks confirm, for example, that variables are declared and types match. Code generation produces object code, and optimisation makes it faster or smaller. Languages such as Java and C# compile to intermediate code, bytecode or CIL, which is not tied to one processor. A virtual machine or runtime on each platform then interprets it or compiles it just in time into native code, combining portability with good speed.

Generic, special purpose, custom-built and off-the-shelf

Generic software can be used for many different tasks without being changed: the same word processor can produce a letter, an invoice or a set of instructions. Because it suits so many users it is written once and sold off-the-shelf in large numbers. Special purpose software is designed for one particular job or business need, such as a system that receives fault reports from machines and sends engineers to repair them. Some special purpose packages are sold to a whole sector, while others are custom-built, also called bespoke or tailor-made, either by in-house developers or by a software house hired by the client. Custom-built software does exactly what is needed, contains no unnecessary features and is supported by developers who know it well. However, it is expensive, takes a long time to deliver, may contain bugs that only show up under particular conditions, and there is little help available online. Off-the-shelf software is available immediately, costs less because development is shared among many buyers, has been tested by large numbers of users and comes with plenty of online support. On the other hand, it may need time to configure, may include features that are never used and may lack something the organisation requires.

Platforms and ways of delivering software

A platform is the base that software runs on: the hardware, particularly the type of processor, the operating system, or the combination of both. Windows and macOS have long run on similar Intel processors yet count as different platforms, because each provides a different application programming interface. Traditionally, a program written for one platform had to be rewritten for another. Cross-platform approaches reduce this effort. Java compiles to bytecode for a virtual processor, and a Java Virtual Machine on each platform runs that same bytecode. Web applications go further: most processing happens on a remote server and the user interface runs in a browser, so any device with a modern browser can use the application. Software reaches users in two main ways. It can be downloaded as an installation package that sets the program up on the device so it runs on local resources, works offline and is paid for by a one-off fee or by a subscription that disables it if not renewed. Alternatively it can be used through the web, running on the publisher's servers, needing little local power but a reliable internet connection, receiving updates centrally and usually charged by renewable subscription.

Proprietary and open source software

Proprietary software, sometimes called closed source, is owned by a person or company and protected by copyright. Users buy a licence to use it, but the source code is not released and the licence forbids copying or modifying it. In return, users usually receive regular updates, documentation and professional technical support, and the products tend to be polished and easy to use. The disadvantages are cost, dependence on the vendor for fixes and new features, paying for features that are not needed, and the risk that support ends if the company stops trading. Freeware is proprietary software given away at no charge; it is still closed source and its licence still has restrictions. Open source software is distributed with its source code under a licence that allows anyone to use, study, change and share it. Copyleft licences such as the GNU GPL also require that modified versions are distributed with their source code. Open source is usually free and not tied to one developer, so development can continue if the original authors stop. However, it may be less thoroughly tested or documented, support often comes from community forums, and several competing versions may exist. Linux and Android's underlying system are well-known examples.

Batch and real-time processing

In batch processing, transactions are gathered over a period into a transaction file and then processed together as one job at a scheduled time, often overnight when the system is quiet. The same operations are carried out on every record, and once the job has been set up it runs without anyone needing to intervene. Batch totals and control totals check that no data has been lost or added. Batch processing is efficient and cheap, and suits jobs where the result is only needed periodically, such as payroll, gas and electricity bills, monthly credit card statements and end-of-day stock reordering. Its weakness is that the master file is out of date until the batch has run. In real-time processing, each transaction updates the master file as soon as it occurs, so information is always current. Bookings and cash machines, where the user waits a few seconds, are real-time transaction (online) processing; systems reacting to sensors within a guaranteed time, such as traffic lights, are real-time control. Real-time processing is essential where a delay could cause harm or loss, such as preventing a concert seat being sold twice, stopping a customer withdrawing more than their balance, or sounding a heart monitor alarm. Because many requests can arrive together, real-time systems use scheduling algorithms to deal with the most urgent first. They need powerful, reliable, often duplicated hardware, so they cost more.

Devices as data processing systems

An automated teller machine is linked to a bank's computer system. The customer inserts a card into the card reader and types a PIN on the keypad; the PIN is sent, encrypted, to the card issuer's bank to be checked. After the menu choice and amount are entered, the system checks the balance, authorises the withdrawal and deducts the amount immediately. Output comes from the screen, the cash dispenser and the receipt printer. A point of sale terminal is a computerised till that also records stock and orders. Barcode scanners, touchscreens and electronic scales provide input; the system looks up each price in a product database, calculates totals and updates stock levels. With EFTPOS, the card reader and PIN pad send payment details to the customer's bank, which transfers the funds within seconds. Outputs include displays, receipts and warning beeps. A smartphone combines phone and computer features under a mobile operating system such as Android or iOS. Its inputs include the touchscreen, microphone, cameras, fingerprint sensor, motion sensors and GPS, plus data from Wi-Fi and mobile networks. Outputs include the screen, speakers, vibration and NFC for contactless payments.

T Key terms from the mark schemes

Compiler
A translator that converts a whole high-level program into object code before it is executed, producing an executable file.

Interpreter
A translator that translates and executes a high-level program one statement at a time, without producing an executable file.

Assembler
A translator that converts assembly language into machine code.

Intermediate code
Processor-independent code, such as Java bytecode or C# CIL, produced by a compiler and executed by a virtual machine or JIT compiler.
`Credited: Bytecode`

Lexical analysis
The first stage of compilation, in which comments and white space are removed, the source code is converted into tokens and a symbol table is built.

Application software
Programs designed to let the end user carry out specific tasks, such as word processing or booking tickets, accessing the hardware through the operating system.

Generic software
Application software that can be used for many different tasks without modification, such as a word processor or spreadsheet.
`Credited: General purpose software`

Special purpose software
Application software designed to carry out one specific task or meet a particular business need.

Custom-built software
Software written specifically for one organisation's requirements, by in-house developers or a third party.
`Credited: Bespoke / tailor-made software`

Off-the-shelf software
Ready-made software developed for a wide market and sold commercially to many customers.

Utility software
System software that helps to maintain, configure and protect a computer, such as antivirus or backup software.
`Credited: Utility program`

Open source software
Software distributed with its source code under a licence that allows users to view, modify and redistribute it.

Proprietary software
Software owned by an individual or company and distributed under a licence that keeps the source code secret and restricts copying and modification.
`Credited: Closed source software`

Batch processing
Processing in which transactions are collected over a period in a transaction file and processed together as one job at a later time, without user intervention.

Real-time processing
Processing in which each transaction or input is dealt with as soon as it occurs, so the master file is always up to date; real-time control systems also guarantee a response fast enough to affect the next input.

C C# example

`A batch run with a control total (C#)`

```
using System;

int[] stock = { 40, 25, 60 };            // master file
(int item, int qty)[] sales =             // transaction file
    { (0, 3), (2, 10), (0, 5), (1, 4) };
int controlTotal = 22;                    // counted at the tills

int sum = 0;
foreach (var s in sales) sum += s.qty;
if (sum != controlTotal)
{
    Console.WriteLine("Control total mismatch: batch rejected");
    return;
}
foreach (var s in sales) stock[s.item] -= s.qty;
for (int i = 0; i < stock.Length; i++)
    Console.WriteLine($"Item {i}: {stock[i]}");
```

Exam focus

Expect 'describe two advantages and two disadvantages' questions on custom-built and off-the-shelf software. Each point should be a full reason, for instance 'off-the-shelf software is cheaper because the development cost is shared among many buyers', not just 'cheaper'. When asked to recommend software in a scenario, use the details given, such as budget, deadlines or staff programming skills, and finish with a justified choice; a business owner with programming experience could adapt open source code, for example. Do not describe open source as simply 'free': the defining feature is that the source code is available and may be modified. Write 'proprietary', not 'propriety'. For batch processing, credit grouping transactions, processing at a set time without intervention, off-peak running and validation with control totals. For real-time, explain the consequence of delay, such as stock being sold twice or an account going overdrawn. Input, processing and output questions on ATMs, POS terminals and smartphones are marked per example, so give separate, specific items and put each in the right category: a card reader is an input device, checking the PIN is processing, and a receipt is output. Translator questions need paired comparisons, and compilation stages must be named in order.

? Check yourself

1. Describe two differences between a compiler and an interpreter. [4]
2. Describe what happens during the lexical analysis and syntax analysis stages of compilation. [4]
3. A dental practice in Bangor uses a spreadsheet for its accounts and a dedicated package for appointments and patient records. Using these two programs, explain the difference between generic and special purpose software. [4]

Answers: 1. Any two differences, 2 marks each: a compiler translates the whole program before it runs (1) whereas an interpreter translates and executes one line/statement at a time (1) / a compiler produces an executable/object code file (1) whereas an interpreter produces no executable so must translate every time it runs (1) / a compiler reports all errors after compilation (1) whereas an interpreter stops at the first error (1) / compiled programs run faster (1) because interpreted code is translated at run time (1) / compiled code can be distributed without the source code (1) whereas interpreted programs need the source code and interpreter (1) 2. Lexical analysis, any two from: white space and comments removed (1) / code split into tokens such as keywords, identifiers and operators (1) / symbol table of identifiers built (1). Syntax analysis, any two from: tokens checked against the grammar rules of the language (1) / parse/syntax tree built (1) / syntax errors reported (1) 3. Generic software can be used for many different tasks without modification (1), e.g. the spreadsheet could also be used for stock lists or rotas (1) / special purpose software is designed for one specific task or business need (1), e.g. the appointments package only manages dental bookings and patient records (1)